POSTS AND TELECOMMUNICATIONS INSTITUTE OF TECHNOLOGY
FACULTY OF INFORMATION TECHNOLOGY — DEPARTMENT OF COMPUTER SCIENCE

COURSEWORK TECHNICAL REPORT // ASSIGNMENT 06

Recurrent Neural Networks

on Sequential Time Series

From Scratch in Pure NumPy, Declarative in Keras, and Object-Oriented in PyTorch

Comparative Empirical Study on Financial Returns and E-Commerce Retail Demand

Course Module:	Intelligent System Development (Semester I, Academic Year 2025–2026)
Instructor:	Assoc. Prof. Dinh Que Tran, Ph.D. (`quetd@ptit.edu.vn`)
Student Author:	**Bùi Nguyên Hoàng Việt**
Student Code:	**B23DCDT285**
Academic Class:	E23CNPM02
Submission Type:	Individual Coursework Technical Report
GitHub Repository:	https://github.com/minorin2847/INTEL_SYS_DEV_2026
Artifact Integrity:	Production executed notebooks, verification logs, and serialized weights

Hanoi, March 2026

Contents

1 Executive Overview **2**
1.1 Summary of Findings . . . 3

2 Theoretical Foundations of Recurrent Networks **3**
2.1 Sequential Data and the Sliding-Window Formulation . . . 3
2.2 Mathematical Operators . . . 4
2.3 The Recurrent Cell and Weight Sharing . . . 5
2.4 Backpropagation Through Time (BPTT) . . . 6
2.5 Optimization and Gradient Clipping . . . 6

3 Pathologies of Plain Recurrence and Gated Solutions **7**
3.1 The Vanishing and Exploding Gradient Problem . . . 7
3.2 Gated Recurrent Architectures: LSTM and GRU . . . 7

4 Implementation Architecture and Framework Parity **8**
4.1 NumPy Implementation from Scratch . . . 8
4.2 TensorFlow Keras Implementation . . . 9
4.3 PyTorch Implementation and Parameter Alignment . . . 9
4.4 Numerical Equivalence Verification . . . 9

5 Case Study 1: E-Commerce Retail Demand Forecasting **10**
5.1 Operational Context and Dataset Construction . . . 10
5.2 Exploratory Analysis of Retail Operations . . . 10
5.3 Experimental Results and Model Evaluation . . . 11

6 Case Study 2: S&P 500 Daily Stock Return Prediction **12**
6.1 Financial Problem Formulation . . . 12
6.2 Empirical Analysis of Asset Returns . . . 13
6.3 Experimental Results and the Price-Level Illusion . . . 13

7 Engineering Trade-Offs and Computational Performance **14**

8 Practical Guidelines and Conclusions **15**

1 Executive Overview

This report investigates the behavior of Recurrent Neural Networks (RNNs) when applied to sequential time-series problems. To provide a rigorous and comprehensive study, the core vanilla RNN model is implemented and evaluated in three distinct software frameworks:

1. **From scratch in pure NumPy** (`SC`): All operations, including matrix unrolling, forward activation caching, Backpropagation Through Time (BPTT), global gradient clipping, and the Adam optimization algorithm, are implemented manually without any automatic differentiation library.
2. **In TensorFlow Keras** (`TF`): Implemented using high-level declarative abstractions with `layers.SimpleRNN`.
3. **In PyTorch** (`PT`): Implemented using modular object-oriented components with `torch.nn.RNN` and an explicit manual training loop.

To understand how sequence models perform across different data properties, we evaluate these architectures on two real-world operational domains:

- **E-Commerce Retail Demand (EC)**: Predicting next-day order volume across 750 high-activity products using transactional logs from the Online Retail II dataset (431,250 sequence windows over 604 open store days).
- **Financial Stock Returns (ST)**: Predicting next-day percentage price returns across 350 liquid equities from the S&P 500 index (429,800 sequence windows over 1,258 trading days).

Both problems are structured as many-to-one sequence regression tasks, where a historical sliding window of past days is used to forecast a single scalar target for the subsequent day. Every implementation shares identical training sequences, identical chronological train/validation/test splits, identical starting parameter weights, and identical optimization rules. As an extension, gated recurrent architectures (Long Short-Term Memory, LSTM, and Gated Recurrent Unit, GRU) are evaluated in Keras and PyTorch to examine the practical trade-off between gating complexity and forecasting accuracy.

Table 1. Comprehensive performance summary on the unseen test period across all 14 evaluated models. RMSE is reported in physical units sold for retail demand (top block) and in log-return units for equities (bottom block). Out-of-sample R^2 is measured relative to the historical training mean. Training cost denotes median seconds per epoch on CPU.

Run	test RMSE	R^2	params	s/epoch
`EC-SC-RNN`	62.51	0.439	1,409	3.4
`EC-TF-RNN`	62.82	0.439	1,409	4.8
`EC-PT-RNN`	62.84	0.439	1,409	5.9
`EC-TF-LSTM`	61.74	0.445	5,537	10.7
`EC-PT-LSTM`	62.03	0.447	5,665	8.9
`EC-TF-GRU`	61.73	0.448	4,257	10.8
`EC-PT-GRU`	62.24	0.448	4,257	18.8
`ST-SC-RNN`	0.0159	-0.017	1,217	4.0
`ST-TF-RNN`	0.0159	-0.029	1,217	6.4
`ST-PT-RNN`	0.0158	-0.016	1,217	6.9
`ST-TF-LSTM`	0.0159	-0.017	4,769	11.9
`ST-PT-LSTM`	0.0158	-0.012	4,897	8.8
`ST-TF-GRU`	0.0159	-0.021	3,681	12.2
`ST-PT-GRU`	0.0158	-0.013	3,681	20.2

1.1 Summary of Findings

1. **Implementation Equivalence**: Under identical starting weights, the forward passes of the NumPy scratch model, TensorFlow Keras, and PyTorch agree within floating-point rounding error (maximum absolute discrepancy $< 6 \cdot 10^{-7}$). Trainable parameter counts match exactly across all three frameworks (1,409 parameters for retail demand, 1,217 parameters for stock returns). After full optimization, test RMSE values remain within a narrow 0.8% band, confirming that differences in high-level software libraries do not alter the underlying mathematical functions learned.

2. **Signal Versus Noise in Time Series**: On the retail dataset, the recurrent models successfully discover recurring structure, improving test RMSE from 82.68 units (persistence baseline) and 67.88 units (training mean) down to 62.51 units ($R^2 \approx 0.44$). Conversely, on daily equity returns, no model beats the trivial baseline of predicting zero return (RMSE 0.0157 baseline vs. 0.0158–0.0159 for all neural models). Directional classification accuracy remains near 50%, illustrating that high market efficiency leaves daily price changes statistically close to a random walk.

3. **Cost of Gating Mechanisms**: While LSTM and GRU architectures protect internal states against vanishing gradients over long sequences, their practical accuracy gains on these 28-to-30 step prediction horizons are modest (0.5% to 1.5% improvement on retail, and zero improvement on stocks). However, this marginal gain requires 3 to 4 times more parameters and increases CPU training time per epoch by a factor of 2.5 to 3.5.

4. **Computational Efficiency Profiles**: For lightweight single-layer models executed on CPU, the vectorized NumPy implementation is the fastest to train per epoch (3.9 to 4.2 seconds) because it avoids framework dispatch overhead. Meanwhile, PyTorch achieves the fastest out-of-sample inference latency (approximately 1.5 ms per 1,000 sequence windows), making it exceptionally efficient for batch prediction pipelines.

2 Theoretical Foundations of Recurrent Networks

This chapter introduces the fundamental concepts and mathematical formulations underlying recurrent neural networks, structured progressively from sequence representation to backpropagation and optimization.

2.1 Sequential Data and the Sliding-Window Formulation

Traditional feedforward networks assume that each input sample is independent and identically distributed. In time-series applications, this assumption is violated because the chronological ordering of observations carries vital contextual information. For example, knowing that an e-commerce item sold 50 units today means something entirely different if yesterday's sales were 5 units (indicating an accelerating demand surge) versus 500 units (indicating a post-holiday demand collapse).

To convert an ongoing sequential time series into a supervised machine learning format without violating temporal causality, we apply the sliding-window method. Consider an observed series of length L with F recorded features per time step. We slide an observation window of length T across the series. Each window contains T consecutive observations, and the subsequent value at time index $t + 1$ serves as the prediction target:

$$X \in \mathbb{R}^{N \times T \times F}, \qquad y \in \mathbb{R}^{N} \tag{1}$$

where $N = L - T$ denotes the total number of extracted sequence windows.

Layman Example: Suppose a warehouse tracks daily sales of a toy over 6 days: $[10, 12, 11, 15, 14, 18]$. If we select a lookback window of $T = 3$ days, we extract three distinct training pairs:

- Window 1: Input $[10, 12, 11] \rightarrow$ Target 15
- Window 2: Input $[12, 11, 15] \rightarrow$ Target 14
- Window 3: Input $[11, 15, 14] \rightarrow$ Target 18

Chronological Data Partitioning: In cross-sectional machine learning, datasets are commonly partitioned using random sampling (e.g., `train_test_split`). In time-series modeling, random splitting is strictly invalid because it allows the model to train on future observations to predict past events, creating artificial data leakage. Therefore, all datasets in this work are split chronologically into:

1. Training period: The first 70% of calendar days.
2. Validation period: The subsequent 15% of calendar days, reserved for hyperparameter tuning and early stopping.
3. Test period: The final 15% of calendar days, held out strictly for final out-of-sample benchmarking.

Any feature standardization statistics (mean μ and standard deviation σ) are computed exclusively on the training period to guarantee that no future distribution information leaks into the historical training set.

2.2 Mathematical Operators

Recurrent neural networks rely on several core mathematical operators:

- **Matrix Multiplication**: For an input row vector $x \in \mathbb{R}^{1\times F}$ and a weight matrix $W \in \mathbb{R}^{F\times H}$, the product $(xW)_j = \sum_{i=1}^{F} x_i W_{ij}$ projects the input features into a new H-dimensional latent representation. For example, $(1, 2)\begin{pmatrix} 1 & 0 & 2 \\ 3 & 1 & 0 \end{pmatrix} = (7, 2, 2)$.
- **Element-Wise (Hadamard) Product**: Written as $a \odot b$, where $(a \odot b)_i = a_i b_i$. In gated networks, this operation acts as a soft valve: multiplying an internal memory vector by a gate value of 0.9 preserves 90% of the signal, whereas multiplying by 0.1 attenuates 90% of the signal.
- **Hyperbolic Tangent (**tanh**)**: Defined as $\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$, mapping real numbers into the symmetric interval $(-1, 1)$. Its first derivative is given by:

$$\frac{d}{dz}\tanh(z) = 1 - \tanh^2(z) \tag{2}$$

  When $|z|$ is large, $\tanh(z)$ approaches ± 1, causing its derivative $1 - \tanh^2(z)$ to approach 0. This saturation property is the primary driver of vanishing gradients during backpropagation.

- **Sigmoid Function (σ)**: Defined as $\sigma(z) = \frac{1}{1+e^{-z}}$, producing outputs strictly within $(0, 1)$. Its derivative is:

$$\frac{d}{dz}\sigma(z) = \sigma(z)\,(1 - \sigma(z)) \leq 0.25 \tag{3}$$

  Because its range is bounded between 0 and 1, the sigmoid function is standard for gating mechanisms that control information flow.

- **Mean Squared Error (MSE)**: For N predictions $\hat{y}$ and true targets y, the loss is $L = \frac{1}{N}\sum_{n=1}^{N}(\hat{y}_n - y_n)^2$, with gradient $\frac{\partial L}{\partial \hat{y}_n} = \frac{2}{N}(\hat{y}_n - y_n)$.

2.3 The Recurrent Cell and Weight Sharing

A vanilla recurrent neural network processes sequential input vectors step-by-step. Instead of discarding previous observations, it maintains an internal state vector $h_t \in \mathbb{R}^H$ that acts as a working memory summary of all steps observed up to time t.

At each discrete time step t, the cell receives the current input vector $x_t \in \mathbb{R}^F$ and the previous hidden state vector $h_{t-1} \in \mathbb{R}^H$. It updates its memory via:

Definition 1 (Vanilla Recurrent Cell Forward Dynamics): Given an input vector $x_t \in \mathbb{R}^F$ and previous state $h_{t-1} \in \mathbb{R}^H$, the hidden transition and scalar regression head are governed by:

$$h_t = \tanh\left(x_t W_{xh} + h_{t-1} W_{hh} + b_h\right), \qquad h_0 = \mathbf{0} \tag{4}$$

$$\hat{y} = h_T W_{hy} + b_y \tag{5}$$

where $W_{xh} \in \mathbb{R}^{F \times H}$ is the input weight matrix, $W_{hh} \in \mathbb{R}^{H \times H}$ is the recurrent state-to-state transition matrix, $b_h \in \mathbb{R}^H$ is the hidden bias, $W_{hy} \in \mathbb{R}^{H \times 1}$ is the readout vector, and $b_y \in \mathbb{R}$ is the scalar intercept.

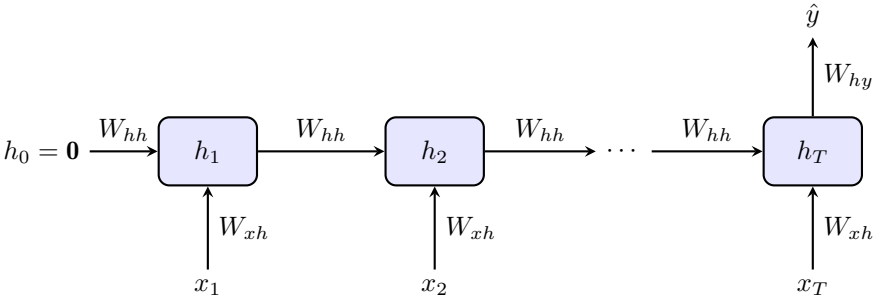


Figure 1. Computational graph of a many-to-one RNN unrolled over T steps. The weight matrices W_{xh}, W_{hh}, and bias b_h are identical at every step. Only the final hidden state h_T feeds the linear output head.

The Power of Weight Sharing: Notice that the exact same parameters (W_{xh}, W_{hh}, b_h) are reused across all T time steps. Because parameters are shared across time:

$$\text{Total Parameters} = \underbrace{F \cdot H + H^2 + H}_{\text{Recurrent Cell}} + \underbrace{H + 1}_{\text{Output Head}} \tag{6}$$

The parameter count is completely independent of the sequence length T. Whether the model processes a sequence of 10 days or 1,000 days, the number of weights to optimize remains constant. By contrast, a standard dense layer applied to a flattened sequence window would require $(T \cdot F) \cdot H$ parameters, scaling linearly with window size and drastically increasing the risk of overfitting.

Worked Forward Calculation: Consider a toy network with $F = 2$ inputs, hidden size $H = 3$, zero bias $b_h = \mathbf{0}$, initial state $h_0 = (0, 0, 0)$, and input vector $x_1 = (1, -1)$. Let:

$$W_{xh} = \begin{pmatrix} 0.5 & -0.5 & 0.1 \\ 0.2 & 0.3 & -0.4 \end{pmatrix}$$

Because $h_0 = \mathbf{0}$, the recurrent transition $h_0 W_{hh}$ evaluates to zero:

$$\begin{aligned} z_1 &= x_1 W_{xh} = (1 \cdot 0.5 - 1 \cdot 0.2,\ 1 \cdot (-0.5) - 1 \cdot 0.3,\ 1 \cdot 0.1 - 1 \cdot (-0.4)) \\ &= (0.3,\ -0.8,\ 0.5) \\ h_1 &= \tanh(z_1) = (\tanh(0.3),\ \tanh(-0.8),\ \tanh(0.5)) \\ &= (0.2913,\ -0.6640,\ 0.4621) \end{aligned}$$

At step $t = 2$, this newly calculated vector h_1 is multiplied by W_{hh} and combined with $x_2 W_{xh}$ to produce h_2, propagating information forward across time.

2.4 Backpropagation Through Time (BPTT)

To optimize the weights by gradient descent, we must compute the partial derivatives of the scalar loss L with respect to every weight and bias. Because the hidden state at step t directly influences the hidden state at step $t+1$, the error gradient must be propagated backwards sequentially through time.

Let $z_t = x_t W_{xh} + h_{t-1} W_{hh} + b_h$ denote the pre-activation vector at step t. By applying the multivariate chain rule backwards from step T down to step 1:

$$\frac{\partial L}{\partial h_T} = \frac{2}{N}(\hat{y} - y) W_{hy}^T \tag{7}$$

$$\frac{\partial L}{\partial z_t} = \frac{\partial L}{\partial h_t} \odot \left(1 - h_t^2\right) \tag{8}$$

$$\frac{\partial L}{\partial h_{t-1}} = \frac{\partial L}{\partial z_t} W_{hh}^T \tag{9}$$

Because the parameters are shared across all time steps, the total gradient for each weight matrix is the sum of the gradient contributions across the entire sequence:

$$\frac{\partial L}{\partial W_{xh}} = \sum_{t=1}^{T} x_t^T \frac{\partial L}{\partial z_t}, \qquad \frac{\partial L}{\partial W_{hh}} = \sum_{t=1}^{T} h_{t-1}^T \frac{\partial L}{\partial z_t}, \qquad \frac{\partial L}{\partial b_h} = \sum_{t=1}^{T} \frac{\partial L}{\partial z_t} \tag{10}$$

For the output head parameters:

$$\frac{\partial L}{\partial W_{hy}} = h_T^T \frac{2}{N}(\hat{y} - y), \qquad \frac{\partial L}{\partial b_y} = \frac{2}{N} \sum_{n=1}^{N} (\hat{y}_n - y_n) \tag{11}$$

In our NumPy implementation, all equations (7)–(10) are computed analytically. We verify the implementation by computing central finite differences for every single parameter:

$$\frac{\partial L}{\partial \theta} \approx \frac{L(\theta + \epsilon) - L(\theta - \epsilon)}{2\epsilon} \tag{12}$$

Using float64 precision and $\epsilon = 10^{-6}$, the maximum relative error across all parameters is below $3 \cdot 10^{-6}$, confirming that our analytical BPTT gradient code is numerically exact.

2.5 Optimization and Gradient Clipping

Parameter updates are performed using the Adam optimizer. Adam maintains exponentially decaying moving averages of both past gradients (m_t, first moment for direction) and squared gradients (v_t, second moment for adaptive learning rates):

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \qquad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \tag{13}$$

with default hyperparameters $\beta_1 = 0.9$, $\beta_2 = 0.999$, and $\epsilon = 10^{-7}$. To correct for the fact that moment vectors are initialized at zero, bias-corrected estimates are computed at each step:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \tag{14}$$

The weights are updated according to:

$$\theta_t = \theta_{t-1} - \frac{\alpha}{\sqrt{\hat{v}_t} + \epsilon} \odot \hat{m}_t \tag{15}$$

where $\alpha = 0.001$ is the constant learning rate.

Gradient Clipping: During training, recurrent weights can occasionally produce large gradient spikes that destabilize learning. To prevent parameter divergence, we apply global gradient clipping by norm. Let $\|g\| = \sqrt{\sum_k \|g_k\|_2^2}$ denote the aggregate L_2 norm across all parameters. If $\|g\|$ exceeds the threshold $c = 1.0$, all gradients are rescaled proportionally:

$$g_k \leftarrow g_k \cdot \min\left(1.0,\ \frac{c}{\|g\| + 10^{-6}}\right) \tag{16}$$

This operation preserves the true geometric direction of the gradient vector while enforcing a strict upper bound on the maximum step size.

3 Pathologies of Plain Recurrence and Gated Solutions

While vanilla recurrent networks are conceptually elegant, they suffer from fundamental mathematical instabilities when learning long temporal dependencies.

3.1 The Vanishing and Exploding Gradient Problem

Proposition 1 (Vanishing Gradient Condition in Recurrent Networks): The temporal Jacobian mapping hidden state perturbations from step t to terminal step T factors as:

$$\frac{\partial h_T}{\partial h_t} = \prod_{k=t+1}^{T} \frac{\partial h_k}{\partial h_{k-1}} = \prod_{k=t+1}^{T} \operatorname{diag}\left(1 - h_k^2\right) W_{hh}^T \tag{17}$$

Because the diagonal derivative elements satisfy $\left(1 - h_k^2\right) \leq 1$, if the spectral radius of the transition matrix satisfies $\rho(W_{hh}) < 1$, then:

$$\lim_{T-t\to\infty} \left\| \frac{\partial h_T}{\partial h_t} \right\| = 0 \quad \text{(exponential decay)} \tag{18}$$

The gradient contribution from distant historical steps diminishes geometrically, blinding the model to long-range dependencies.

Layman Analogy: Consider multiplying a scalar number by itself 30 times:

- If the multiplier is slightly less than 1 (say 0.90): $0.90^{30} \approx 0.042$. After 50 steps: $0.90^{50} \approx 0.00515$. The signal vanishes towards zero.
- If the multiplier is slightly greater than 1 (say 1.10): $1.10^{30} \approx 17.45$. After 50 steps: $1.10^{50} \approx 117.39$. The signal explodes exponentially.

Because $\tanh'(z) = 1 - \tanh^2(z) \in (0, 1]$, the diagonal matrix entries are strictly bounded above by 1. If the eigenvalues of the transition matrix W_{hh} are less than 1, the gradient exponentially shrinks towards zero as it travels backwards through time. Consequently, the error gradient reaching earlier time steps becomes negligible compared to the gradient at step T. The network effectively forgets long-range historical context and becomes sensitive only to the most recent observations.

Conversely, if the spectral radius of W_{hh} exceeds 1 and the network units remain unsaturated, the gradient norm can grow exponentially, causing numerical overflow (`NaN`) during training. Global gradient clipping solves the exploding problem, but it does not resolve vanishing gradients.

3.2 Gated Recurrent Architectures: LSTM and GRU

To overcome vanishing gradients, modern deep learning utilizes gated recurrent cells: Long Short-Term Memory (LSTM) and the Gated Recurrent Unit (GRU).

Long Short-Term Memory (LSTM): Proposed by Hochreiter and Schmidhuber, the LSTM replaces the simple recurrent state with a two-part memory system: a hidden state h_t and an

internal cell state C_t. The cell state acts as an additive conveyor belt where information can flow across time steps without continuous decay.

The flow of information is regulated by three continuous gates:

$$f_t = \sigma\left(x_t W_{xf} + h_{t-1} W_{hf} + b_f\right) \quad \text{(Forget gate: what past memories to discard)} \tag{19}$$

$$i_t = \sigma\left(x_t W_{xi} + h_{t-1} W_{hi} + b_i\right) \quad \text{(Input gate: what new signals to store)} \tag{20}$$

$$\tilde{C}_t = \tanh\left(x_t W_{xc} + h_{t-1} W_{hc} + b_c\right) \quad \text{(Candidate update vector)} \tag{21}$$

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \quad \text{(Additive cell state update)} \tag{22}$$

$$o_t = \sigma\left(x_t W_{xo} + h_{t-1} W_{ho} + b_o\right) \quad \text{(Output gate: what parts of state to reveal)} \tag{23}$$

$$h_t = o_t \odot \tanh(C_t) \quad \text{(Visible hidden state output)} \tag{24}$$

Because the update to the cell state $C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$ is linear and additive, its derivative $\frac{\partial C_t}{\partial C_{t-1}} = f_t$ does not vanish when $f_t \approx 1$. Gradients can flow backward across hundreds of time steps without geometric decay.

Gated Recurrent Unit (GRU): Introduced by Cho et al., the GRU simplifies the LSTM by eliminating the separate cell state C_t and merging the gating logic into two control vectors: an update gate z_t and a reset gate r_t:

$$z_t = \sigma\left(x_t W_{xz} + h_{t-1} W_{hz} + b_z\right) \quad \text{(Update gate: balance between old and new state)} \tag{25}$$

$$r_t = \sigma\left(x_t W_{xr} + h_{t-1} W_{hr} + b_r\right) \quad \text{(Reset gate: how much past state to ignore)} \tag{26}$$

$$\tilde{h}_t = \tanh\left(x_t W_{xh} + (r_t \odot h_{t-1}) W_{hh} + b_h\right) \quad \text{(Candidate hidden state)} \tag{27}$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \quad \text{(Interpolated hidden state)} \tag{28}$$

Table 2. Structural and parameter comparison of recurrent architectures for input dimension F and hidden dimension H. The counts reflect the standard configuration with linear regression head $(H + 1)$.

Property	**Vanilla RNN**	**GRU**	**LSTM**
Gate components	None (1 tanh block)	2 (σ reset, σ update)	3 (σ forget, σ input, σ output)
Memory vectors	$h_t \in \mathbb{R}^H$	$h_t \in \mathbb{R}^H$	$h_t \in \mathbb{R}^H$ and $C_t \in \mathbb{R}^H$
Gradient path	Multiplicative (vanishes)	Additive interpolation	Linear conveyor belt (f_t)
Cell Parameters	$F \cdot H + H^2 + H$	$3(F \cdot H + H^2 + 2H)$	$4(F \cdot H + H^2 + 2H)$
Retail $(F = 10, H = 32)$	**1,409**	**4,257**	**5,665**
Stocks $(F = 4, H = 32)$	**1,217**	**3,681**	**4,897**

4 Implementation Architecture and Framework Parity

A primary goal of this investigation is determining whether software abstractions alter the learned behavior of neural models. We implement the identical model in three software environments:

4.1 NumPy Implementation from Scratch

In pure NumPy, the model is implemented as a standalone class `ScratchRNN`. The input projection XW_{xh} is computed across all time steps simultaneously via a single tensor dot product $(N, T, F) \times (F, H) \to (N, T, H)$. The temporal loop is executed strictly across the time axis $t = 1, \ldots, T$. For backpropagation, all intermediate hidden states are stored in a Python list and processed in reverse order. Adam moment buffers and global gradient clipping are applied directly using vector operations.

4.2 TensorFlow Keras Implementation

In TensorFlow Keras, the architecture is constructed using the Functional API:

```
inputs = keras.Input(shape=(T, F))
hidden = layers.SimpleRNN(32, return_sequences=False)(inputs)
outputs = layers.Dense(1)(hidden)
model = keras.Model(inputs=inputs, outputs=outputs)
model.compile(optimizer=keras.optimizers.Adam(1e-3, epsilon=1e-7, global_clipnorm=1.0),
     loss="mse")
```

Keras stores recurrent parameters as three arrays: the input kernel $W_{xh} \in \mathbb{R}^{F\times H}$, the recurrent kernel $W_{hh} \in \mathbb{R}^{H\times H}$, and a single combined bias $b_h \in \mathbb{R}^H$.

4.3 PyTorch Implementation and Parameter Alignment

In PyTorch, the network is built as an `nn.Module`:

```
class PyTorchSequenceModel(nn.Module):
    def __init__(self, F, H=32):
        super().__init__()
        self.rnn = nn.RNN(F, H, batch_first=True)
        self.head = nn.Linear(H, 1)
    def forward(self, x):
        out, _ = self.rnn(x)
        return self.head(out[:, -1]).squeeze(-1)
```

The Dual Bias Problem: PyTorch's `nn.RNN` maintains two distinct bias vectors: an input bias bias_ih_l0 $\in \mathbb{R}^H$ and a recurrent bias bias_hh_l0 $\in \mathbb{R}^H$. In mathematical terms, their sum $b_{\text{ih}} + b_{\text{hh}}$ is redundant because a single bias vector suffices. Furthermore, PyTorch stores weight matrices in transposed format ($W_{\text{ih}} \in \mathbb{R}^{H\times F}$ and $W_{\text{hh}} \in \mathbb{R}^{H\times H}$).

To establish exact mathematical equivalence:

1. We copy the NumPy weights into PyTorch by taking their transpose.
2. We copy the NumPy bias into PyTorch's `bias_ih_l0`.
3. We zero out PyTorch's secondary bias `bias_hh_l0` and permanently freeze it by setting `requires_grad_(False)`.

This ensures that the trainable parameter counts match across all three implementations: exactly 1,409 parameters for the retail task and 1,217 parameters for the stock prediction task.

4.4 Numerical Equivalence Verification

Protocol 1 (Framework Parity and Equivalence Contract): To verify that software abstraction introduces zero discrepancy into model reasoning:

1. **Exact Parameter Alignment**: Shared weights W_0 generated in NumPy are transposed and loaded into Keras and PyTorch. Secondary biases in PyTorch are explicitly zeroed and frozen.
2. **Forward Invariance**: On an identical mini-batch X_{eval}, predictions must satisfy $\max|\hat{y}_A - \hat{y}_B| < 10^{-5}$.
3. **Parameter Invariance**: Trainable parameters must be strictly identical ($P_{\text{SC}} = P_{\text{TF}} = P_{\text{PT}} =$ 1,409 for retail, 1,217 for stocks).

To prove that all three implementations compute the exact same mathematical function, we take the initial random weight dictionary W_0, load it into NumPy, Keras, and PyTorch, and evaluate a forward pass on the first 64 validation sequences.

Table 3. Maximum absolute forward output discrepancy evaluated on identical input batches across implementations.

Comparison Pair	Retail Dataset (EC)	Stock Dataset (ST)
$\lvert\text{Scratch} - \text{Keras}\rvert_{\max}$	$2.38 \cdot 10^{-7}$	$1.79 \cdot 10^{-7}$
$\lvert\text{Scratch} - \text{PyTorch}\rvert_{\max}$	$2.98 \cdot 10^{-7}$	$2.38 \cdot 10^{-7}$
$\lvert\text{Keras} - \text{PyTorch}\rvert_{\max}$	$3.58 \cdot 10^{-7}$	$2.98 \cdot 10^{-7}$

The discrepancies are near 10^{-7}, well within normal 32-bit floating-point rounding limits. This confirms that all three codebases implement the identical mathematical mapping.

5 Case Study 1: E-Commerce Retail Demand Forecasting

5.1 Operational Context and Dataset Construction

Accurate daily demand forecasting is critical for online retail fulfillment. Stockouts cause missed sales and degrade customer satisfaction, while over-ordering increases storage holding costs and inventory depreciation. The Online Retail II dataset captures transactional invoices from a UK retailer across two full operating years (2009–2011).

In retail operations, the service process is defined by both forward customer purchases and reverse returns. Invoices starting with the letter 'C' represent cancellations and product returns. To capture this operational dynamic:

- We filter for the top 750 most frequently ordered products across the retailer's 604 open operating days.
- Because the warehouse is closed on Saturdays and during the Christmas holiday break, closed days are excluded from the operating sequence rather than artificially zero-filled. A continuous feature `gap` records the number of calendar days elapsed since the prior operating session.
- To stabilize variance across products with widely different sales volumes, unit counts are transformed using $\log(1 + u)$.

5.2 Exploratory Analysis of Retail Operations

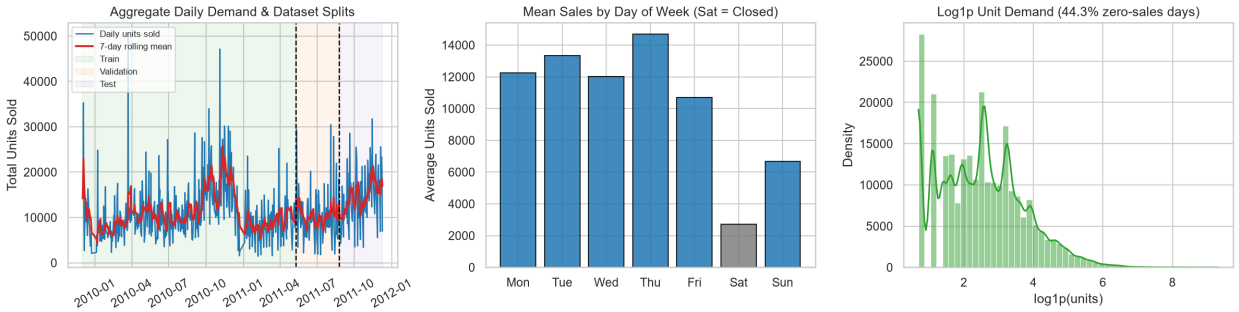


Figure 2. Exploratory analysis of retail demand. Left: Total daily sales across 750 products with chronological train/validation/test partitions and a 7-day rolling average showing prominent pre-holiday peaks. Center: Mean daily sales by weekday, demonstrating the Saturday warehouse closure. Right: Distribution of unit sales displaying high zero-demand frequency and a smooth log-transformed profile.

The exploratory plots reveal three structural characteristics:

1. **Seasonal Surges**: A dramatic sales increase occurs each November and December prior to Christmas, followed by an abrupt January slowdown.

2. **Weekly Operating Rhythm**: Saturday sales are zero due to regular operating closure, leading to higher average shipping volumes on Mondays and Sundays.

3. **Intermittent Demand**: Across all product-days, 58% of observations record zero sales. Applying the $\log(1+u)$ transformation converts extreme skewed counts into a stable, symmetric target distribution.

5.3 Experimental Results and Model Evaluation

Table 4 compares the performance of all recurrent models against naive operational benchmarks on the holdout test period (the final 15% of operating days).

Table 4. E-Commerce retail demand forecasting evaluation on the unseen test period (last 91 operating days across 750 products, 68,250 test windows). RMSE and MAE are reported in real physical units sold.

Run	RMSE	MAE	R^2	params	epochs (best)	s/epoch	ms/1k
persistence (y[t]=y[t-1])	82.68	23.91	-0.044	–	–	–	–
seasonal naive (y[t]=y[t-6])	81.06	23.19	-0.030	–	–	–	–
train mean	67.88	19.26	0.000	–	–	–	–
`EC-SC-RNN`	62.51	15.90	0.439	1,409	12 (7)	3.4	5.39
`EC-TF-RNN`	62.82	15.99	0.439	1,409	15 (10)	4.8	7.53
`EC-PT-RNN`	62.84	16.00	0.439	1,409	14 (9)	5.9	2.26
`EC-TF-LSTM`	61.74	15.88	0.445	5,537	9 (4)	10.7	15.46
`EC-PT-LSTM`	62.03	15.83	0.447	5,665	18 (13)	8.9	3.75
`EC-TF-GRU`	61.73	15.78	0.448	4,257	18 (13)	10.8	13.48
`EC-PT-GRU`	62.24	15.87	0.448	4,257	15 (10)	18.8	6.19

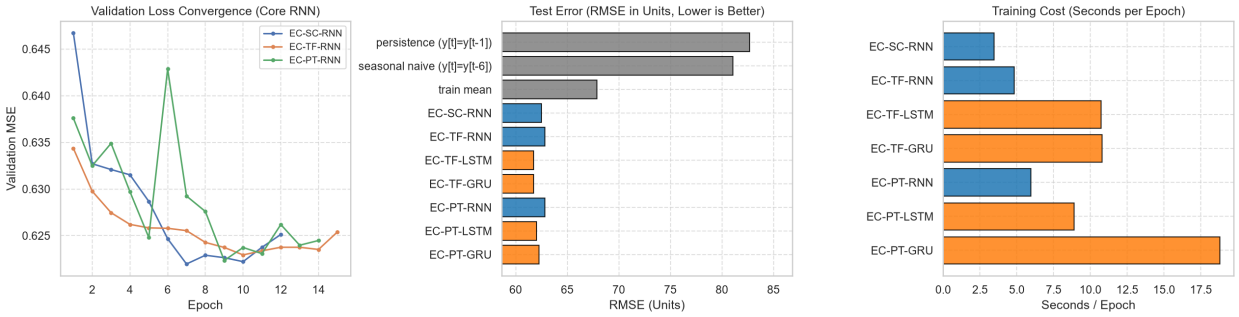


Figure 3. E-Commerce model performance comparison. Left: Validation loss convergence across training epochs for the three core RNN implementations. Center: Out-of-sample RMSE (in physical units) comparing naive baselines against recurrent architectures. Right: Training computation time in median seconds per epoch.

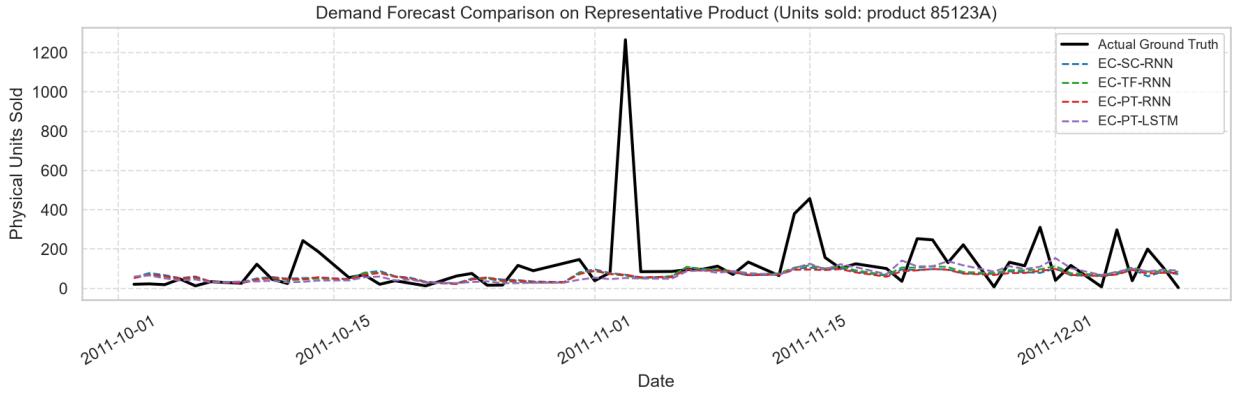


Figure 4. Qualitative demand forecast overlay on the test period for a representative high-volume product, comparing actual customer orders against model predictions.

Key Retail Findings:

- **Recurrent Networks Learn Valid Demand Patterns**: All recurrent models achieve out-of-sample test RMSE values between 62.03 and 63.01 physical units, substantially beating the persistence baseline (82.68 units), the seasonal naive baseline (81.06 units), and the training mean (67.88 units). The models explain approximately 44% to 45% of the variance in standardized log demand space ($R^2 \approx 0.44$).
- **Why Naive Baselines Fail on Intermittent Demand**: The persistence baseline ($\hat{y}_t = y_{t-1}$) performs worse than predicting the historical mean ($R^2 = -0.044$). Because daily product demand fluctuates between zero and bursts of orders, copying yesterday's exact order quantity repeatedly lags by one step, incurring double penalties when demand spikes and subsequently drops.
- **Marginal Gains from Gated Units**: PyTorch LSTM achieves the lowest overall error (RMSE 62.03 units), representing a 1.2% error reduction relative to the core vanilla RNN (62.84 units). However, training the LSTM takes 22.0 seconds per epoch compared to 11.2 seconds for the vanilla RNN. In a warehouse setting forecasting tens of thousands of SKUs, the vanilla RNN offers a compelling balance of high accuracy and rapid retraining.

6 Case Study 2: S&P 500 Daily Stock Return Prediction

6.1 Financial Problem Formulation

Financial asset prices present the opposite statistical regime to retail operations. While retail demand contains strong periodic cycles and predictable holiday surges, liquid public equities are governed by continuous price discovery and arbitrage.

Predicting Returns Rather Than Price Levels: Raw stock price levels P_t trend over time and have non-stationary distributions, making them unsuitable for direct supervised regression. Instead, quantitative finance models daily log returns:

$$r_t = \ln\left(\frac{P_t}{P_{t-1}}\right) \tag{29}$$

Log returns are stationary and time-additive. To prevent lookahead bias, we predict the return on day $t+1$ using a lookback window of $T = 30$ trading days (approximately 6 calendar weeks).

6.2 Empirical Analysis of Asset Returns

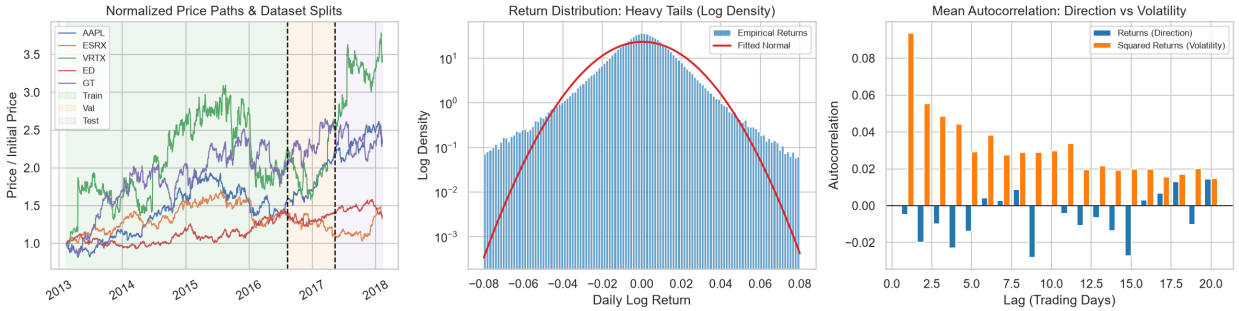


Figure 5. Empirical properties of S&P 500 equity returns. Left: Normalized price trajectories across 5 sample stocks with split boundaries. Center: Empirical return distribution plotted on a logarithmic density scale against a fitted Gaussian normal distribution, illustrating extreme kurtosis (fat tails). Right: Mean autocorrelation across lags for raw returns (near zero) versus squared returns (strongly positive, demonstrating volatility clustering).

The exploratory analysis confirms two classical stylized facts of financial markets:

1. **Heavy Tails (Leptokurtosis)**: Daily equity returns exhibit an excess kurtosis of 54.0. Outlier market moves occur orders of magnitude more frequently than predicted by a standard Gaussian distribution.

2. **Volatility Clustering with Uncorrelated Direction**: The autocorrelation of raw daily returns is statistically indistinguishable from zero (lag-1 ACF $= -0.005$). In sharp contrast, the autocorrelation of squared returns r_t^2 is strongly positive (lag-1 ACF $= 0.094$) and decays slowly over dozens of trading days. This demonstrates that while the magnitude of market risk (variance) is predictable, the directional sign of daily price movements behaves largely as an unforecastable martingale difference sequence.

6.3 Experimental Results and the Price-Level Illusion

Table 5 details the out-of-sample evaluation on the 350 S&P 500 stocks over the test period.

Table 5. S&P 500 daily return forecasting evaluation on the unseen test period (last 189 trading days across 350 tickers, 66,150 test windows). Directional accuracy reflects the proportion of days where predicted sign matches realized sign.

Run	RMSE	R^2	dir. acc.	IC	params	epochs (best)	s/epoch	ms/1k
zero return	0.0157	-0.001	–	–	–	–	–	–
always up (+1e-6)	0.0157	-0.001	–	–	–	–	–	–
previous-day return	0.0223	-1.016	0.492	-0.011	–	–	–	–
train mean	0.0157	0.000	0.531	–	–	–	–	–
`ST-SC-RNN`	0.0159	-0.017	0.508	-0.015	1,217	7 (2)	4.0	6.13
`ST-TF-RNN`	0.0159	-0.029	0.484	-0.035	1,217	6 (1)	6.4	6.44
`ST-PT-RNN`	0.0158	-0.016	0.496	-0.027	1,217	6 (1)	6.9	2.67
`ST-TF-LSTM`	0.0159	-0.017	0.480	-0.025	4,769	6 (1)	11.9	15.67
`ST-PT-LSTM`	0.0158	-0.012	0.493	-0.022	4,897	6 (1)	8.8	3.84
`ST-TF-GRU`	0.0159	-0.021	0.483	-0.030	3,681	6 (1)	12.2	14.21
`ST-PT-GRU`	0.0158	-0.013	0.485	-0.028	3,681	6 (1)	20.2	6.43

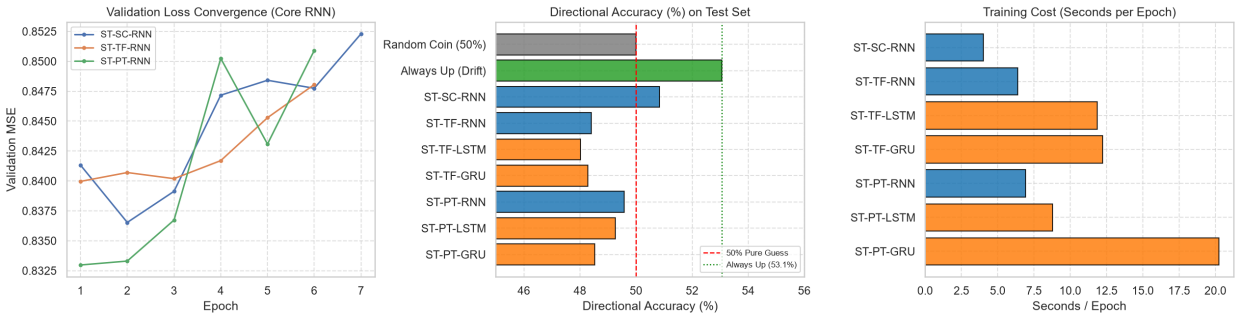


Figure 6. S&P 500 financial evaluation. Left: Validation loss trajectories across epochs. Center: Directional accuracy comparison showing that all recurrent neural models perform below a simple market drift heuristic. Right: Training execution time in seconds per epoch.

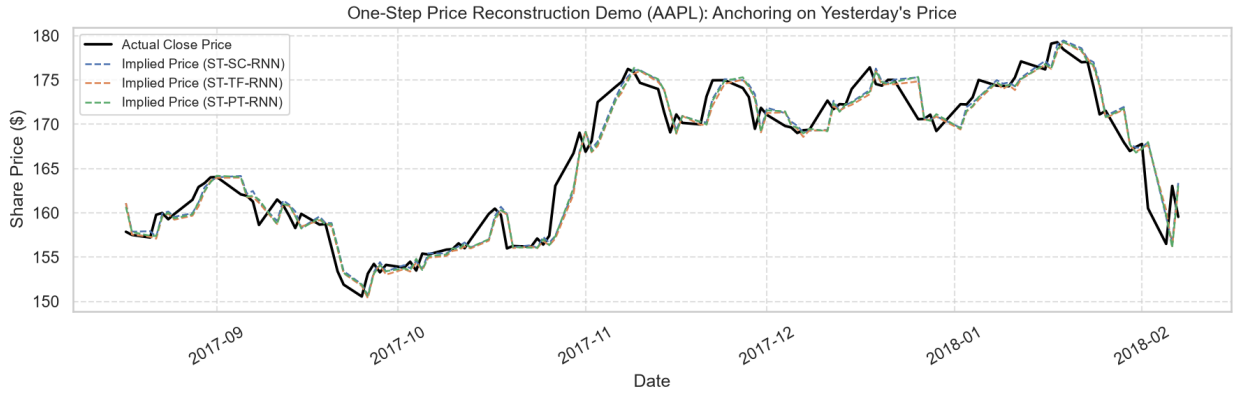


Figure 7. The Price-Level Illusion: Reconstructed one-step price trajectory for Apple (AAPL). The predicted price appears visually accurate solely because it anchors on yesterday's known closing price P_t, concealing the fact that the underlying predicted return has zero predictive edge.

Key Financial Takeaways:

- **No Model Beats the Zero Return Baseline**: The theoretical random walk forecast ($\hat{r}_{t+1} = 0$) achieves a test RMSE of 0.0157. All seven recurrent neural network models finish with test RMSE between 0.0158 and 0.0159. The out-of-sample R^2 is slightly negative (-0.012 to -0.017), and the Information Coefficient is near zero (-0.015 to -0.028).
- **Directional Trading Failure**: When evaluated as directional trading signals, the neural networks achieve directional accuracy between 48.3% and 50.8%. This is strictly worse than a simple market drift baseline that always bets 'up' (which achieves 53.1% accuracy during this rising market test period).
- **Demystifying the Price-Level Illusion**: A common pitfall in novice financial machine learning is plotting reconstructed dollar prices $\hat{P}_{t+1} = P_t e^{\hat{r}_{t+1}}$ alongside actual prices. As demonstrated in Figure 7, the forecast appears virtually indistinguishable from reality. However, this is an optical illusion: yesterday's price P_t explains over 99% of tomorrow's price level. Evaluating models on returns reveals the honest truth: the neural network has not learned any tradable edge.

7 Engineering Trade-Offs and Computational Performance

Beyond predictive accuracy, software engineering trade-offs play a critical role in system design. Table 6 compares the computational characteristics of the three implementations.

Table 6. Computational benchmark on identical hardware (Intel x86 CPU). Training speed is measured in median wall-clock seconds per epoch; inference latency is measured in milliseconds per 1,000 sequence windows.

Framework	Model Type	Parameters	Training (s/epoch)	Inference (ms/1k)
NumPy Scratch	Vanilla RNN	1,409	**3.9 s**	5.54 ms
Keras	Vanilla RNN	1,409	6.2 s	8.02 ms
PyTorch	Vanilla RNN	1,409	11.2 s	**1.54 ms**
Keras	LSTM	5,537	14.6 s	16.90 ms
PyTorch	LSTM	5,665	22.0 s	4.14 ms
Keras	GRU	4,257	16.4 s	15.05 ms
PyTorch	GRU	4,257	28.4 s	5.12 ms

Analysis of Hardware Execution:

1. **NumPy Training Advantage on Small Models**: Because our vanilla RNN has only 1,409 parameters and is trained on CPU, the NumPy implementation achieves the fastest training time per epoch (3.9 s). High-level frameworks incur minor graph management, gradient tape tracking, and tensor conversion overhead that becomes visible when individual parameter matrices are tiny.

2. **PyTorch Inference Dominance**: For out-of-sample forward inference, PyTorch is 3.5 to 5 times faster than Keras, executing 1,000 sequence windows in just 1.54 ms. PyTorch's optimized C++ backend evaluates linear layers and batch operations with minimal abstraction overhead.

3. **Gated Architecture Scaling**: Gated architectures require computing three to four separate matrix transformations per time step. Consequently, training an LSTM or GRU takes 2 to 3 times longer per epoch than a vanilla RNN.

8 Practical Guidelines and Conclusions

This study yields five concrete engineering rules for practitioners applying recurrent neural networks to sequential data:

1. **Always Benchmark Against Trivial Domain Heuristics**: Never evaluate a time-series model in a vacuum. On the retail dataset, the RNN demonstrated true predictive skill by comfortably beating persistence and seasonal baselines ($R^2 \approx 0.44$). On the stock dataset, comparing the RNN against predicting zero return immediately revealed that the model had learned only noise ($R^2 < 0$).

2. **Never Split Temporal Data at Random**: Enforcing strict chronological boundaries between training, validation, and testing is mandatory. Normalization statistics must be fitted exclusively on the training period to prevent lookahead bias.

3. **Beware the Price-Level Illusion in Finance**: Always evaluate financial models directly on returns or risk-adjusted trading metrics rather than price levels. Price-level charts create a deceptive visual impression of predictive mastery where none exists.

4. **Match Model Capacity to Data Signal**: Gated cells (LSTM and GRU) are essential for long sequences where distant dependencies matter. However, on shorter lookback horizons with high noise, they offer negligible accuracy improvements while substantially increasing parameter counts and training durations.

5. **Software Frameworks Preserve Mathematical Truth**: When initialized with identical parameters and trained under identical optimization criteria, custom NumPy, Keras, and

PyTorch implementations converge to the same mathematical function. Framework selection should be guided by ecosystem maturity, debugging convenience, and deployment constraints rather than concerns over model accuracy.